

Implementation of a TPM-Based Trusted Boot Chain with LUKS Disk Encryption and CI/CD Integration on Fedora/RHEL

Student: Timur Khakimov

PXL supervisor: Guido Frissaer

Workplace supervisors: Merijn Lowis & Indy van Mol

Piros, Diestsesteenweg 695, 3010 Leuven (Kessel-Lo)

Associate Degree in Systems and Network Administration (SNA)

Abstract

Context: Piros is a Belgian IT consultancy with fewer than 30 employees and is a recognized Red Hat Premier Business Partner. The company helps customers design, manage, and secure Linux-based infrastructure, automating wherever possible. These environments process sensitive and confidential data every day.

Problem: common security controls such as firewalls, user permissions, and network monitoring only take effect after the system has booted. They therefore provide no protection when the boot process itself has been compromised. An attacker who manipulates the boot chain can start the operating system in an untrusted state without making that compromise visible. Such an attack can also undermine disk encryption at the moment the user enters the decryption credential. The research question of this graduation project is therefore: how can Piros design, implement, and test a hardware-backed trusted-boot system based on a TPM, UKIs, and Policy-Based Key Release so that only correctly booted Linux systems can access encrypted data?

Approach: the research addresses this question in six consecutive phases on Fedora and Red Hat Enterprise Linux (RHEL). First, a virtual test environment is created with UEFI Secure Boot and a vTPM on Proxmox. UKIs are then generated and cryptographically signed so that systemd-boot accepts only validated images. Next, the TPM measurements in the PCRs are analysed to establish the complete boot sequence. Policy-Based Key Release is then connected to LUKS disk encryption on the basis of those measurements. An existing CI/CD pipeline is also extended to support reproducible image builds, signing, and testing. Finally, attack scenarios, including changes to kernel parameters and the initramfs, are executed to evaluate the resilience of the solution.

Result: the research delivers a working and reproducible trusted boot chain, validated on Fedora Server 43 with portability to RHEL as a design objective. The disk-encryption key is released only when the machine boots into an approved, unmodified state. The concrete deliverables are a technical implementation guide with step-by-step CLI instructions and an analysis of the attack vectors that the solution does and does not address, tested through focused attack scenarios. Piros consequently receives a directly usable foundation for adding this technology to its future security offerings.

List of Abbreviations

Abbreviation	Full term	Brief description
CI/CD	Continuous Integration / Continuous Delivery	Automated build and delivery pipeline.
CLI	Command Line Interface	Text-based command-line environment.
db	Signature Database	Collection of keys whose signatures are accepted by firmware.
DMA	Direct Memory Access	Direct memory access by peripheral devices; a relevant attack class.
DNF	Dandified YUM	Package manager used by Fedora and RHEL.
EDK II	EFI Development Kit II	UEFI reference firmware (TianoCore), used as virtual firmware in the test environment.
ELF	Executable and Linkable Format	Binary format for Linux executables; relevant to the objcopy finding.
ESP	EFI System Partition	FAT partition containing the boot loader and UKIs.
FAT	File Allocation Table	File system used by the ESP.
HSM	Hardware Security Module	Device for secure, centralized key management.
KEK	Key Exchange Key	Secure Boot key that authorizes modifications to db.
LUKS	Linux Unified Key Setup	Linux standard for disk encryption.
MID	Machine ID	Unique machine identifier used as the prefix of the UKI filename.
OLR	Program-Specific Learning Outcome	Learning outcome defined by the degree program.
OS	Operating System	Operating system.
PBKR	Policy-Based Key Release	Release of a key by the TPM on the basis of a PCR policy.
PCR	Platform Configuration Register	Append-only TPM register containing a measurement of a boot component.
PE+	Portable Executable (Plus)	Executable binary format loaded by UEFI; a

		UKI is a PE+ binary.
PK	Platform Key	Highest-level Secure Boot key; authorizes modifications to the KEK.
RHEL	Red Hat Enterprise Linux	Commercial Linux distribution with long-term support.
RPM	RPM Package Manager	Package format and management system used by Fedora and RHEL.
RSA	Rivest-Shamir-Adleman	Asymmetric cryptographic algorithm used for the policy key.
SBAT	Secure Boot Advanced Targeting	Revocation mechanism for Secure Boot components.
SDG	Sustainable Development Goal	Sustainable Development Goal of the United Nations.
TPM	Trusted Platform Module	Security chip that manages boot measurements and cryptographic keys.
UEFI	Unified Extensible Firmware Interface	Firmware interface between hardware and the operating system.
UKI	Unified Kernel Image	A single signed UEFI binary containing the kernel, initramfs, and command line.
VM	Virtual Machine	Virtual machine.
VMA	Virtual Memory Address	Address field in a PE+ section; relevant to the objcopy finding.
vTPM	Virtual TPM	Software TPM used in the virtual test environment.

Table of Contents

Table of Contents.....	2
1 Company Profile.....	4
2 Project Question, Research Activities, and Results.....	5
2.1 Problem Context.....	5
2.2 Research Question and Subquestions.....	6
2.3 Research Activities.....	7
2.3.1 Subquestion 1 (shared) and Subquestion 2 (own): Secure Boot, UKI Signing, and TPM Measurements.....	8
2.3.2 Subquestion 6 (own): Updates and Preservation of the <i>Secure Boot Chain</i>	9
2.3.3 Subquestion 7 (own): <i>Security Value</i> and Attack Scenarios.....	10
2.4 Collected Results.....	11
3 Final Result, Conclusions, and Recommendations.....	12
3.1 Final Result.....	12
3.2 Conclusions.....	17
3.3 Recommendations.....	18
4 Personal Reflections.....	19
4.1 Personal Reflection.....	19
4.1.1 AIM Reflection 1: The Technical Process.....	19
4.1.2 AIM Reflection 2: Personal Development, X-Factor, and OLRs.....	20
4.2 Reflection on the Sustainable Development Goals.....	22
4.2.1 Potential Positive Contributions.....	22
4.2.2 Potential Unintended Negative Impact.....	23
5 References.....	24
6 Appendices.....	26

1 Company Profile

Piros is a Belgian IT consultancy based in Leuven (Kessel-Lo). It serves organizations that process sensitive and confidential data, including customers in finance, the public sector, and large enterprises. In Belgium, Piros operates in the niche of IT service providers specializing in open-source infrastructure and security. At European and global level, the company is a local part of the wider Red Hat partner ecosystem, which is active in the same fields.

Position in the organization. Because Piros is a small company with fewer than thirty employees, it does not use a formal organizational chart divided into separate departments. The graduation project and workplace learning took place within the company's general consultancy activities under the supervision of workplace coaches Merijn Lowis and Indy van Mol. Technical coordination was performed directly with the workplace coaches and the managing director, without intermediate departmental structures.

Unique selling points and IT culture. Piros positions itself as a Red Hat technology specialist and holds Red Hat Premier Business Partner status. Within the Red Hat partner program, this is the highest level of technical recognition and provides access to the full Red Hat portfolio, including Red Hat Enterprise Linux, OpenShift, Ansible Automation Platform, and related products. The company focuses primarily on enterprise Linux infrastructure rather than Windows-oriented environments. Its technical methodology includes infrastructure as code with Ansible, containerization with Podman, and version control and CI/CD with GitLab. Its IT culture emphasizes open source, reproducible and declarative infrastructure, and direct technical dialogue with customers.

Specific organizational data. Piros has fewer than thirty employees, all working in IT consultancy. Its annual accounts are publicly available through the National Bank of Belgium and show strong growth: revenue increased from approximately EUR 5.3 million in 2021 to approximately EUR 17.0 million in 2024, while staffing grew from approximately 8.5 to 18.8 full-time equivalents over the same period. Piros obtained Red Hat Premier status on the basis of both the number of Red Hat-certified consultants and the scale of Red Hat implementations delivered to customers. This status is relevant to the graduation project because the trusted boot chain was designed for migration from Fedora to Red Hat Enterprise Linux. The work validated on Fedora Server 43 should therefore be transferable to the RHEL distributions supported by Piros without major architectural changes.

2 Project Question, Research Activities, and Results

This chapter first defines the problem, explains the research question and its subquestions, describes the research activities, and presents their results. The graduation project was conducted by two

students, with a formal allocation of 105 hours per student. This report covers the work assigned to Timur Khakimov: designing and validating a Secure Boot key hierarchy and building signed Unified Kernel Images (UKIs), analysing TPM measurements in Platform Configuration Registers (PCRs), automating the update path through a kernel-install hook, and carrying out attack scenarios against the completed boot chain. The tasks assigned to fellow student Matteo Ideler Cautaeert - the test environment, Policy-Based Key Release, LUKS integration, and the CI/CD pipeline - are mentioned where required to explain the overall design, but the detailed justification of those components is contained in his report.

During implementation, the work was adjusted at several points compared with the original project proposal and plan of approach. These changes were not cosmetic; they affected the architecture of the signing mechanism itself. Section 2.3 documents the adjustments alongside the relevant activities so that the reasons why the final solution differs from the original proposal remain clear.

2.1 Problem Context

IT consultancies such as Piros operate systems that process sensitive and confidential information every day. Conventional controls such as firewalls, access rights, network monitoring, and endpoint detection only become effective after the operating system has booted. If an attacker has already compromised the boot chain, for example by introducing a modified kernel or initramfs, all of those controls run on an untrusted foundation. The problem becomes even more serious when an encrypted disk is unlocked on the compromised system: the user enters the passphrase and thereby supplies the key to an environment whose integrity has not been established. This is the classic evil-maid scenario, in which an attacker with physical access silently alters a device, for example in a hotel room, so that the user discloses the key or password to a manipulated environment at the next boot.

Modern hardware provides a combination of mechanisms that can mitigate this problem. UEFI Secure Boot verifies the cryptographic signature of the code it loads. The Trusted Platform Module (TPM) records measurements of each loaded boot component in its Platform Configuration Registers (PCRs). LUKS2 supports a keyslot whose release of the volume encryption key can be tied to those PCR values. LUKS2 is the current version of Linux Unified Key Setup, the standard Linux full-disk-encryption format: an encrypted volume stores multiple keyslots, each of which unlocks the same master key through a different method, such as a password or TPM policy. In Linux, these building blocks are brought together through the Unified Kernel Image (UKI), a single UEFI executable in which the kernel, initramfs, and kernel command line are signed and measured as one object. The individual components are documented upstream, but no single guide integrates Secure Boot, UKIs, TPM measurements, Policy-Based Key Release, and LUKS into a working Fedora or Red Hat Enterprise Linux solution.

This technology is operationally relevant to Piros. macOS and Windows currently provide more integrated hardware-bound boot integrity, while equivalent Linux functionality is still evolving. The required components exist, but they are not yet delivered as a single ready-made feature in the Red Hat distributions supported by Piros. Developing the integration now allows Piros to build expertise before customers request it. The result is therefore a validated reference implementation and reproducible runbook rather than a final, permanent guide, because the underlying technology continues to change. Piros faces two related questions: a technical one - whether a working trusted

boot chain can be built on the platforms it supports and how - and an operational one - what security value it adds and which attack classes remain outside its scope.

2.2 Research Question and Subquestions

The shared principal research question is:

How can Piros design, implement, and test a hardware-bound trusted-boot system based on a TPM, Unified Kernel Images, and Policy-Based Key Release so that only correctly booted Linux systems can access encrypted data?

The principal question was divided into subquestions that structure the project. Subquestions within the student's own scope are marked with an asterisk. The original proposal contained seven subquestions. Subquestions 3 and 4 were addressed by both students from different perspectives: the student from the operator perspective and the fellow student from the CI/CD perspective. Because the student's work expanded beyond the original plan, an eighth subquestion was added on fail-closed automation of the signing chain. Subquestion 6 was formally shared but was in practice answered largely through the student's kernel-install hook, and is therefore also marked with an asterisk.

- Subquestion 1: How can Secure Boot be configured so that only signed Unified Kernel Images can boot? (Shared: the fellow student provides the test environment; the student provides the key hierarchy and signing chain.)
- (*) **Subquestion 2:** How can TPM measurements and PCRs be used in practice to validate boot integrity?
- (*) **Subquestion 3:** How can Policy-Based Key Release be used to release encryption keys only for a valid system state? (Own work from the operator perspective: the student seals the policy private key to PCR 7 and supplies the forward-sealed PCR 11 signature. The fellow student answers the same question from the CI/CD perspective.)
- (*) **Subquestion 4:** How can LUKS disk encryption be technically linked to TPM measurements? (Own work from the operator perspective: the student binds a LUKS2 keyslot through systemd-cryptenroll to the combined PCR 7 and PCR 11 policy. The fellow student addresses the same integration from the CI/CD perspective.)
- Subquestion 5: How can a CI/CD pipeline automatically generate a trusted custom Linux build (UKI)? (Fellow student.)
- (*) **Subquestion 6:** How does the completed operating system handle updates, and how is the Secure Boot chain preserved after an OS update?
- (*) **Subquestion 7:** What is the exact security value of this setup, and against which attack vectors or system compromises is it effective?
- (*) **Subquestion 8:** How can signing and forward-sealing of the boot chain be automated fail-closed across arbitrary package transactions, not only kernel updates, including the systemd-boot loader binary, without operator intervention?

2.3 Research Activities

For each subquestion within the student's scope, the following sections describe the concrete activities, the research method used, and, where applicable, the adjustment from the original plan. Subquestions assigned to the fellow student are covered in his report. The integration between both parts - specifically, how the student's signing chain produces the PCR values used by the fellow student's keyslot binding - is described in Section 2.4.

Mapping to the original plan of approach. Table 1 compares the activities proposed in the plan of approach within the student's scope with what was actually delivered, identifying where adjustment was required or the work expanded beyond the original plan. The activities are discussed in detail in the following sections.

Planned activity (plan of approach)	Completed activity	Status / adjustment
Generate custom Secure Boot keys and enroll them in firmware	PK/KEK/db generated; factory keys removed; enforcing mode active	Completed as planned
Build and sign a UKI with ukify	Manual Phase 1 build with ukify build and an Authenticode signature; validated through a UEFI boot	Completed as planned
Analyse TPM measurements in PCRs	PCR 7 and PCR 11 read; runtime values compared with the systemd-measure prediction	Completed as planned
Build pipeline with Podman / Buildah (image-based)	kernel-install hook (drop-in script in /etc/kernel/install.d/)	Adjusted: package-managed update path selected instead of an image path (see Activity 5 in Section 2.3)
Automate UKI rebuilding during a kernel update	Hook produces a freshly signed UKI with an embedded .pcrsig section; validated through dnf reinstall kernel-core	Completed through an adjusted implementation path
Store the TPM policy private key securely	Key sealed with systemd-creds encrypt against PCR 7	Completed as planned
Execute attack scenarios against the boot chain	Scenarios for replacement of vmlinuz, initramfs, command line, and the UKI; rejection verified at	Being finalized at submission time

	firmware or LUKS level	
Validate the intermediate signing chain through sbverify	Validation through sbverify, pesign, and a real VM boot	Extended: an additional firmware-validation step was added after finding that sbverify alone is insufficient
(Not included in the plan of approach)	Two fail-closed automation chains based on libdnf5-plugin-actions: a DNF decider/executor that detects boot-input drift and re-signs the UKIs through the hook, and a separate chain that re-signs the systemd-boot loader and propagates it to the ESP	Extended beyond the plan; documented as Subquestion 8 (Activity 8)

Table 1: Mapping of planned activities to completed activities.

2.3.1 Subquestion 1 (shared) and Subquestion 2 (own): Secure Boot, UKI Signing, and TPM Measurements

Activity 1: generate a Secure Boot key hierarchy and enroll it in firmware. A complete Secure Boot hierarchy - Platform Key, Key Exchange Key, and signature database - was generated locally rather than relying on the factory and Microsoft keys installed by default. This gives the lab direct ownership of the chain of trust. The factory keys were removed from firmware and the shim layer was omitted from the Fedora boot chain. Research method: literature review of the UEFI specification [11] and Fedora documentation [16], followed by a proof-of-concept implementation in a Proxmox test environment with a software TPM and snapshots before and after every irreversible step.

Activity 2: build a signed UKI with ukify and validate it through firmware. A UKI combines the kernel, initramfs, command line, and os-release data in a single UEFI PE+ executable that systemd-boot loads directly [1]. It is therefore a single signed .efi application that firmware could in principle load directly. In this setup, the fixed chain is UEFI -> signed systemd-boot -> signed UKI: systemd-boot remains the boot loader that selects and launches the UKI. The UKI was built with ukify build, and the Authenticode signature was applied in the same invocation using the custom db key [4]. Research method: technical implementation and layered validation using sbverify for the signature, pesign for the signer count, and finally a real boot in the virtual machine. The final step proved essential, as explained below.

Activity 3: analyse TPM measurements in PCR 7 and PCR 11. During boot, systemd-stub inside the UKI measures each relevant section - kernel, initramfs, cmdline, osrel, uname, sbat, and pcrpkey - separately into PCR 11. Firmware populates PCR 7 with the Secure Boot policy [3]. Runtime values were read from /sys/class/tpm/tpm0/pcr-sha256/ and compared with the values predicted in advance by systemd-measure calculate using those same UKI sections [6]. Research method: implementation, observation, measurement, and comparison of predicted and runtime PCR values.

Activity 4: generate a policy keypair and seal the private key to PCR 7 through the TPM. The private key used to sign future PCR 11 predictions must not remain on disk in plaintext. An RSA keypair was therefore generated. The public key was made available to systemd-cryptenroll as the future trust anchor stored in the LUKS header [8]. The private key was sealed with systemd-creds encrypt against PCR 7 [9]. It can consequently be decrypted only on this machine and only while Secure Boot is operating under the unchanged policy state. Research method: implementation and cryptographic validation of the seal-unseal cycle.

2.3.2 Subquestion 6 (own): Updates and Preservation of the Secure Boot Chain

Activity 5: design and validate a kernel-install hook. Every DNF kernel update must produce a new UKI, sign it, and issue a forward-sealed PCR 11 signature for that UKI without operator intervention. A kernel-install drop-in was created for this purpose at /etc/kernel/install.d/80-tpm2-sign.install with mode 0755 [5]. The hook receives the paths to the new vmlinuz and initramfs from kernel-install, unseals the policy private key from the TPM, builds a fresh UKI with ukify build, predicts PCR 11 for the enter-initrd phase through systemd-measure calculate, signs that prediction with the policy key, and atomically places the resulting UKI, including its embedded .pcrsig section, on the ESP at /boot/efi/EFI/Linux/\${MID}-\${KVER}.efi. Research method: design, ShellCheck static analysis, dry-run tests with synthesized KERNEL_INSTALL_* environment variables, and final validation through a real dnf reinstall kernel-core transaction followed by a reboot.

Adjustment from the plan of approach. The original plan named Podman and Buildah as build tools and assumed a container-oriented, bootc-based path. During implementation it became clear that the update cycle Piros would encounter at customers uses conventional RPM package updates rather than replacement of an entire image. The kernel-install hook was therefore designed for the package-managed update path, not the image path. This was a deliberate change: the resulting architecture fills an operational gap between fully image-based operating systems such as bootc and fragile manual re-enrollment, matching the reality of regulated and operationally traditional environments.

Technical problem 1: objcopy --add-section produces a UKI that sbverify accepts but UEFI rejects. The first hook implementation used objcopy --add-section .pcrsig=... to add the PCR signature after an already fully signed UKI had been produced. The resulting file passed sbverify and contained a valid .pcrsig section, yet UEFI LoadImage() rejected it at boot with Load Error. The plausible cause is that objcopy was designed primarily for ELF files [18] and assigns an abnormal section VMA when rewriting PE+ images, which UEFI interprets as an invalid PE+ layout [12]. Linux-side tools such as sbverify and pesign verify the Authenticode signature but do not validate the PE+ layout in the same way as firmware during actual loading. This led to an important meta-finding not explicitly documented upstream: successful sbverify output is not proof that firmware can load a binary. The fix was structural. The hook now builds the UKI in a single ukify build invocation so that all sections,

including .pcrsig, are constructed by ukify's PE+ pipeline. The failed UKI was retained as a forensic artifact; see the appendices.

Technical problem 2: ukify build --join-pcrsig silently produces no .pcrsig section. An alternative path suggested by upstream documentation, ukify build --join-pcrsig, was found on systemd 258.7 to return successfully without writing a .pcrsig section [4]. The generated UKI therefore lacks the section even though the command reports no error. The fix was again structural: the hook now supplies the native PCR-signing options --pcr-private-key, --pcr-public-key, and --phases=enter-initrd in the same ukify build invocation that also applies the Authenticode signature.

2.3.3 Subquestion 7 (own): Security Value and Attack Scenarios

Activity 6: design and execute attack scenarios against the completed boot chain. A trusted boot chain only has value if it blocks concrete attacks. The following scenarios were developed: modification of the kernel command line, replacement of the initramfs, replacement of vmlinuz, overwriting the UKI with an unsigned alternative, removal of the .pcrsig section, and rollback to an older but still validly signed UKI. For each scenario, the attack is executed in the Proxmox VM, the rejection stage is observed - firmware through LoadImage(), LUKS unlock through a PCR 11 mismatch, or no rejection - and the attack classes outside the solution's scope are documented, such as an attacker with physical access and hardware DMA capabilities. Research method: test-case design, attack simulation, integrity verification, and analysis of results.

Additional activities and evidence beyond the original plan. Several activities and data-collection steps not explicitly listed in the plan of approach proved necessary during implementation. First, a three-way verification procedure for PCR 11 prediction was introduced: manually, within the hook, and through an independent systemd-measure calculate invocation outside the hook. The original plan specified only sbverify, but additional validation layers were required after it became clear that a Linux-side check does not prove firmware acceptance. Second, the hook gained a PE+ VMA sanity check with a threshold of 0x180000000 to detect abnormal layouts early. Third, the failed objcopy UKI, bad-objcopy-pcrsig-loadererror.efi, was preserved as a forensic artifact for future PE+-level research. These additional actions materially improved the result: without the three-way validation a silent error in the signing path could have gone undetected, and without the VMA check a future ukify change might reproduce the failure of the first implementation.

2.3.4 Subquestions 3 and 4 (own): Policy-Based Key Release and LUKS-TPM Integration

Activity 7: bind a LUKS2 keyslot to the PCR policy. The disk-unlock decision was tied to the measured boot state. Through `systemd-cryptenroll`, a LUKS2 keyslot was connected to a split policy: PCR 7 for the static Secure Boot state and a policy-key-signed prediction of PCR 11. The TPM releases the key only when the system boots into the expected state; a different measurement falls back to the recovery passphrase in keyslot 0. The policy private key created in Activity 4 signs the PCR 11 component so that the update path developed for Subquestion 6 does not break the binding. This answers Subquestion 3, release under a valid system state, and Subquestion 4, technical linkage between LUKS and TPM measurements, from the operator perspective. The fellow student covers the equivalent linkage from the CI/CD pipeline perspective. Research method: implementation through `systemd-cryptenroll` followed by a reboot that unlocked without a passphrase.

2.3.5 Subquestion 8 (own): Fail-Closed Automation of the Signing Chain

Activity 8: automate signing fail-closed across package transactions. The kernel-install hook from Activity 5 covers kernel package updates, but not every transaction that changes measured boot inputs. Two independent fail-closed automation chains were therefore created using `libdnf5-plugin-actions` rules, a decider, and an executor. The first chain monitors fourteen categories of boot input, including `dracut` configuration, kernel modules, firmware, and `udev` rules. After every DNF transaction the decider computes a manifest, compares it with a recorded baseline, and invokes the executor only when drift is present. The executor rebuilds and re-signs the UKIs through `kernel-install` and the hook. The second chain re-signs the `systemd-boot` loader binary when the `systemd-boot-unsigned` package changes and propagates it to the ESP. Both chains create a sentinel before changing protected state and remove it only after success, so an interrupted run fails closed. Research method: design, staged gate-by-gate validation with snapshots, and a live transaction followed by a reboot and byte-for-byte comparison of runtime PCR 11. Figure 3 illustrates both chains. Their complete source code is included in Appendices 3b and 3c.

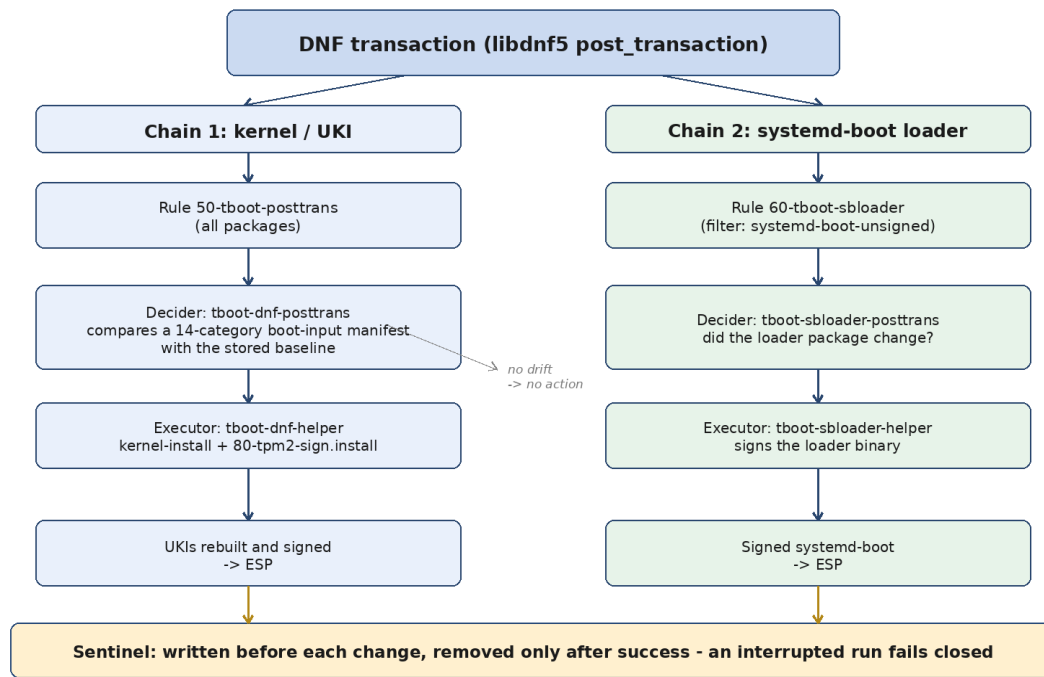


Figure 3: The two fail-closed automation chains. A DNF transaction invokes the `libdnf5 post_transaction` plug-ins. Chain 1, on the left, detects drift in monitored boot inputs and rebuilds and signs the UKIs through `kernel-install` and the `80-tpm2-sign.install` hook. Chain 2, on the right, re-signs the `systemd-boot` loader binary. A sentinel makes every run fail closed.

2.4 Collected Results

The concrete result for each subquestion within the student's scope is summarized below. Table 2 lists the most important measurable results. The following paragraphs then discuss the outcome of each subquestion and conclude with an assessment of whether the collected evidence is sufficient to solve the practical problem.

Subquestion	Collected result	Status
1 (shared)	Custom Secure Boot PK/KEK/db enrolled in firmware; signed UKI accepted by UEFI LoadImage().	Complete
2 (own)	Runtime PCR 7 and PCR 11 values match the systemd-measure prediction byte for byte; three-way verification.	Complete
3 (own)	LUKS2 keyslot bound through systemd-cryptenroll to the combined PCR 7 and PCR 11 policy; the disk unlocks without a passphrase under a valid system state, with passphrase fallback in keyslot 0. (Operator perspective; fellow student provides the CI/CD variant.)	Complete
4 (own)	LUKS disk encryption technically linked to TPM measurements through the PCR-policy keyslot; validated by a reboot without a passphrase. (Operator perspective; fellow student provides the CI/CD variant.)	Complete
6 (own)	After dnf reinstall kernel-core, the kernel-install hook	Complete

	produces a signed UKI with a valid .pcrsig section; reboot successful.	
7 (own)	Attack scenarios executed; rejection verified at firmware or LUKS level; remaining attack classes documented.	Being finalized
8 (own)	Two fail-closed automation chains (kernel/UKI and systemd-boot loader) installed and validated; drift triggers automatic re-signing, interrupted runs fail closed, and runtime PCR 11 equals the prediction byte for byte.	Complete

Subquestion 1: signing chain. A custom Secure Boot PK/KEK/db hierarchy was generated and enrolled in firmware, and the lab boots in Secure Boot enforcing mode without factory keys. A UKI signed through ukify is recognized by systemd-boot and successfully loaded by UEFI. Phase 1 was completed.

Subquestion 2: TPM measurements and PCRs. The boot process populates PCR 7 with the Secure Boot policy and measures the UKI sections into PCR 11 in a fixed order through systemd-stub. systemd-pcrphase services then extend PCR 11 in the sequence enter-initrd -> leave-initrd -> sysinit -> ready [7]. The runtime PCR 11 value matches, byte for byte, the prediction calculated by systemd-measure during the build and embedded in .pcrsig. The prediction was validated three ways: manually, by the hook itself, and by an independent systemd-measure calculate invocation outside the hook.

Subquestion 6: updates. The kernel-install hook was fully validated. A dnf reinstall kernel-core transaction produces a newly signed UKI with a valid .pcrsig section at the correct path; the system then boots directly into that UKI and runtime PCR 11 matches the prediction issued by the hook during the build. The signed UKI was subsequently configured as the persistent systemd-boot default and confirmed through a normal reboot. The complete update path works without operator intervention while preserving the Secure Boot chain.

Subquestion 7: security value and attack scenarios. Attacks against vmlinuz, the initramfs, and the command line are rejected by Secure Boot at LoadImage() level when they invalidate the Authenticode signature. Attacks that alter measured UKI content, such as replacement with another UKI, fail at LUKS-unlock level when systemd-cryptsetup encounters a PCR 11 mismatch. At submission time, the full analysis of residual attack classes was still being finalized; the results and logs were to be included in the appendices.

Subquestions 3 and 4: key release and LUKS integration. A LUKS2 keyslot was bound through systemd-cryptenroll to the combined PCR 7 and PCR 11 policy. When the system boots into the expected state, systemd-cryptsetup unlocks the disk without a passphrase. A different measurement falls back to the recovery passphrase in keyslot 0. The operator-side integration is complete; the fellow student provides the CI/CD-side variant.

Subquestion 8: fail-closed automation. The two automation chains for the kernel/UKI path and the systemd-boot loader were installed and validated. Drift in monitored boot inputs causes an automatic signed rebuild, while an interrupted execution leaves the sentinel in place so the machine is not

rebooted without supervision. After a live transaction, runtime PCR 11 matched the prediction byte for byte.

Are the collected data sufficient to solve the practical problem? The available evidence - firmware acceptance of the signed UKI, byte-for-byte PCR 11 agreement, a successful DNF update followed by reboot, and verified rejection of attack scenarios at firmware or LUKS level - is sufficient to answer the practical question within the student's scope. Two evidence gaps remain. First, the dracut-sensitive update path is covered in the lab by the fail-closed DNF automation chain from Subquestion 8: the decider detects drift and causes the UKIs to be rebuilt and signed. Validation against customer-specific package flows, particularly NVIDIA akmod and kmod workflows and transactions that introduce a new kernel version, remained open and is included in the recommendations. Second, the explanation of the objcopy failure is plausibly supported by the PE+ specification [12] and objcopy behaviour [18], but the exact differing PE+ field was not proven. The collected evidence is sufficient to establish that a working, automatable boot chain can be built; a formal upstream bug report would require additional investigation.

3 Final Result, Conclusions, and Recommendations

This chapter presents the concrete final result of the graduation project, answers the principal research question on the basis of the collected evidence, and formulates recommendations for further operational use at the workplace.

3.1 Final Result

The final result within the student's scope consists of three related deliverables, all technically operational in the test environment. Supporting CLI output is included in Appendix 4, and the findings catalog in Appendix 5. Table 3 lists the principal tools, technologies, and services used to produce the result.

A working and validated UKI signing chain on Fedora Server 43. The chain consists of a custom Secure Boot PK/KEK/db hierarchy manually enrolled in firmware after removing the factory keys, a signed systemd-boot binary, and signed UKIs generated by ukify build. The system boots in enforcing mode with no fallback to the factory trust chain. bootctl status and sbverify output are included in Appendix 4.

A validated kernel-install hook that fully automates the update path. The canonical hook, 80-tpm2-sign.install with mode 0755, builds a fresh UKI for every kernel update, predicts and signs PCR 11 for the enter-initrd phase, and installs the resulting UKI atomically on the ESP. The complete hook source is included in Appendix 3a. It was validated through a real dnf reinstall kernel-core transaction followed by a reboot, with three independent checks of runtime PCR 11 against the prediction. Transaction logs and the PCR comparison are included in Appendix 4.

A forensically documented findings catalog. Findings are organized across boot-path behaviour, UKI construction, TPM and credentials, systemd-measure and ukify, kernel-install and DNF, and tooling. Every finding includes a verdict - FORBIDDEN, CONDITIONAL, or CONTEXT-ONLY - as well as a symptom, root-cause analysis, and correction. Several findings close documentation gaps not explicitly covered in upstream sources. The complete catalog is included in Appendix 5.

Architecture representation. The architecture is illustrated through two diagrams: Figure 1 presents the build-time flow and Figure 2 the boot-time flow. Higher-resolution versions are included in Appendices 1 and 2. The build-time diagram shows dnf upgrade -> kernel-install -> ukify build -> signing with the TPM-sealed policy private key -> signed UKI on the ESP. The boot-time diagram shows UEFI firmware -> Secure Boot verification of the UKI -> systemd-stub measurement of the UKI sections into PCR 11 -> phase extensions by systemd-pcrphase services -> LUKS unlock by systemd-cryptsetup through the PCR 11 policy.

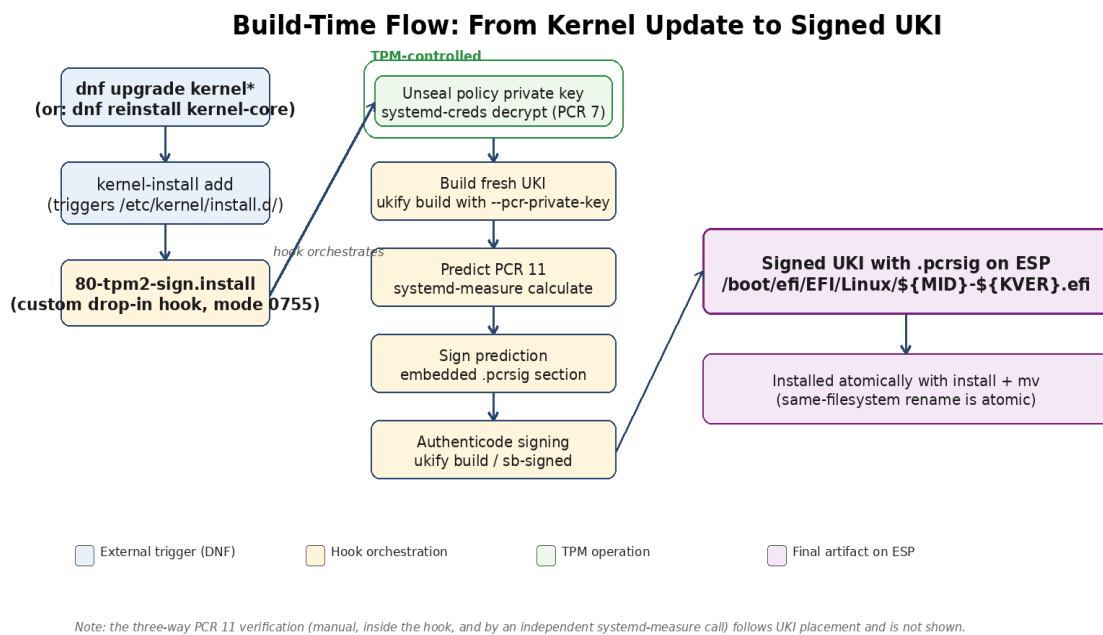


Figure 1: Build-time flow. A kernel update through DNF triggers kernel-install, which invokes the custom 80-tpm2-sign.install hook. The hook unseals the policy private key from the TPM, builds a new UKI with ukify, predicts and signs PCR 11, and atomically places the signed UKI on the ESP.

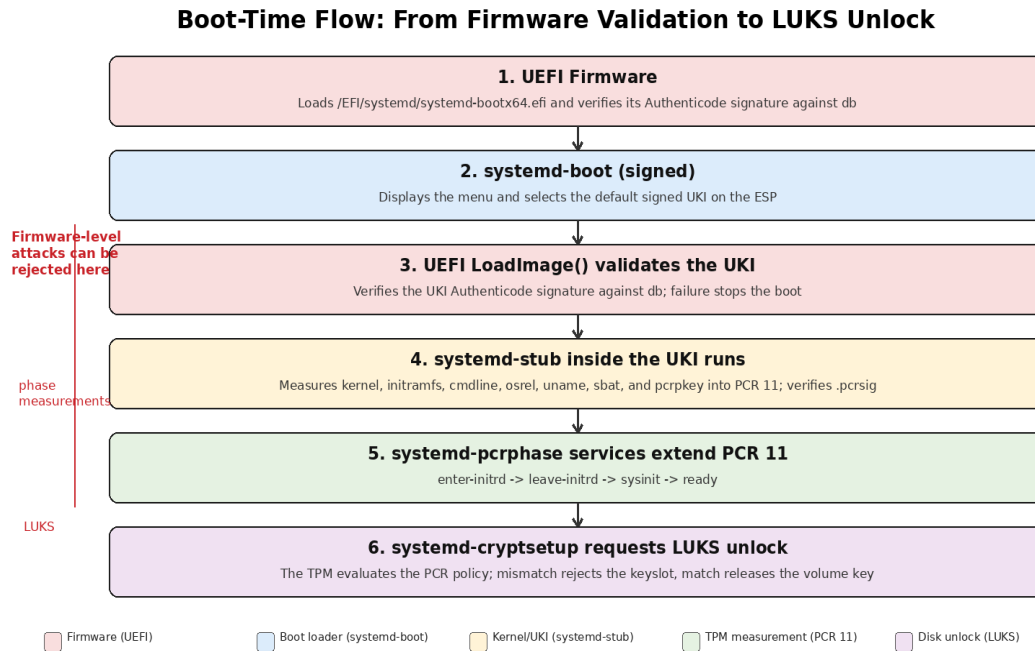


Figure 2: Boot-time flow. UEFI firmware verifies the Authenticode signatures of systemd-boot and the UKI. systemd-stub measures the UKI sections into PCR 11, systemd-pcrphase services extend PCR 11 for each phase, and systemd-cryptsetup requests the TPM policy and unlocks LUKS when the policy matches.

Tool / technology	Function in the boot chain
Fedora Server 43	Production operating system of the test environment
Proxmox VE with software TPM	Virtual test environment with snapshot discipline
UEFI Secure Boot	Firmware layer that verifies the UKI Authenticode signature
systemd-boot	Boot loader that loads the UKI
ukify	Builds PE+ UKIs and inserts Authenticode and .pcrsig sections [4]
kernel-install	Orchestrates UKI rebuilding during kernel package updates [5]
systemd-measure	Predicts PCR 11 values for forward sealing [6]
systemd-pcrphase	Extends PCR 11 for each boot phase during runtime [7]
systemd-cryptenroll	Adds the PCR-policy keyslot to the LUKS header [8]
systemd-creds	Seals the policy private key to PCR 7 in the TPM [9]
sbsigntool / sbverify	Applies and verifies Authenticode signatures [19]
pesign	Independent verification of the number of signatures [20]
efitools	Management of Secure Boot key databases [21]

dracut	Builds the initramfs embedded in the UKI [22]
tpm2-tools / tpm2-tss	TPM user-space tooling for sealing and attestation [23][24]
OpenSSL	RSA keypair generation for the policy key [25]
LUKS2 / cryptsetup	Disk encryption with a PCR-policy keyslot [14][15]
Bash + ShellCheck	Hook implementation language and static analysis
Git	Version control for the runbook, hook source, and evidence

Representative runtime-validation evidence. The following output blocks illustrate the structure of the evidence collected in the test environment. Complete logs with dates and timestamps are included in Appendix 4.

Example 1: bootctl status output after a successful reboot into the signed UKI. Secure Boot is in user mode, and the machine-ID-prefixed UKI is the active default entry.

```
$ bootctl status
System:
  Firmware: UEFI 2.70 (Proxmox distribution of EDK II 1.00)
  Firmware Arch: x64
  Secure Boot: enabled (user)
  TPM2 Support: yes (measured uki)
  Measured UKI: yes
  Boot into FW: supported

Current Boot Loader:
  Product: systemd-boot 258.7-1.fc43
  Features: ✓ Boot counting
            ✓ Menu timeout control
            ✓ One-shot menu timeout control
            ✓ Default entry control
            ✓ Random seed support
            ✓ Boot loader sets ESP information
  ESP: /boot/efi (/dev/sda1)
  File: └─/EFI/BOOT/BOOTX64.EFI
        /EFI/systemd/systemd-bootx64.efi

Default Boot Loader Entry:
  title: Fedora Linux 43 (Server Edition)
  id: <SANITIZED-MACHINE-ID>-6.19.14-200.fc43.x86_64.efi
  source: /boot/efi/EFI/Linux/<SANITIZED-MACHINE-ID>-6.19.14-200.fc43.x86_64.efi
  type: uki
  linux: 6.19.14-200.fc43.x86_64
```

Example 2: comparison between runtime PCR values and the systemd-measure prediction, verified after boot.

```
$ cat /sys/class/tpm/tpm0/pcr-sha256/7
BF8878BD8B70654924BD39CA71D770F5... (complete value in Appendix 4)

$ cat /sys/class/tpm/tpm0/pcr-sha256/11
28E66CE224C9BB0711059470D77103A5... (complete value in Appendix 4)

$ /usr/lib/systemd/systemd-measure calculate \
  --linux=/usr/lib/modules/6.19.14-200.fc43.x86_64/vmlinuz \
  --initrd=/boot/initramfs-6.19.14-200.fc43.x86_64.img \
  --osrel=/etc/os-release \
  --cmdline=$(cat /etc/kernel/cmdline) \
  --phase=enter-initrd
11:sha256=... # signed enter-initrd value, stored in .pcrsig
```

Example 3: the 80-tpm2-sign kernel-install hook runs during a DNF transaction, predicts and signs PCR 11, and atomically places the UKI on the ESP.

```
$ sudo dnf reinstall -y kernel-core
...
Running scriptlet: kernel-core-6.19.14-200.fc43.x86_64
+ kernel-install add 6.19.14-200.fc43.x86_64 \
  /lib/modules/6.19.14-200.fc43.x86_64/vmlinuz
[80-tpm2-sign] entering: COMMAND=add KERNEL_VERSION=6.19.14-200.fc43.x86_64
[80-tpm2-sign] unsealing policy private key from TPM (PCR 7)... ok
[80-tpm2-sign] running ukify build with --pcr-private-key, --phases=enter-initrd ... ok
[80-tpm2-sign] PE+ VMA sanity check (threshold 0x1800000000) ... ok
[80-tpm2-sign] atomic install -> /boot/efi/EFI/Linux/<SANITIZED-MACHINE-ID>-6.19.14-
200.fc43.x86_64.efi
[80-tpm2-sign] hook completed: sha256(hook)=a455444a...3b8c502f
Verifying      : kernel-core-6.19.14-200.fc43.x86_64 ... ok
Complete!
```

Example 4: the tboot-dnf-posttrans decider detects drift in monitored boot inputs after a dracut-sensitive transaction and invokes tboot-dnf-helper, which rebuilds and re-signs the UKI. This is illustrative; the complete logs are in Appendix 4.

```
$ sudo dnf reinstall -y dracut-network
...
Running scriptlet: libdnf5-plugin-actions (post_transaction)
[tboot-dnf-posttrans] computing boot-input manifest (14 categories) ...
[tboot-dnf-posttrans] manifest sha256=7c1f9a... baseline sha256=3a90b2...
[tboot-dnf-posttrans] DRIFT in category 'dracut-modules' -> invoking helper
[tboot-dnf-helper] sentinel set: /var/lib/tboot-dnf-helper/UNSAFE-T0-REBOOT
[tboot-dnf-helper] kernel-install add 6.19.14-200.fc43.x86_64
[80-tpm2-sign] unsealing policy private key from TPM (PCR 7) ... ok
[80-tpm2-sign] ukify build --phases=enter-initrd ... ok
[80-tpm2-sign] atomic install -> /boot/efi/EFI/Linux/<SANITIZED-MACHINE-ID>-6.19.14-
200.fc43.x86_64.efi
[tboot-dnf-helper] baseline advanced; sentinel cleared
Complete!
```

Example 5: after an update of systemd-boot-unsigned, tboot-sbloader-posttrans and tboot-sbloader-helper resign the systemd-boot loader and propagate it to the ESP. sbverify confirms the custom db signature. This is illustrative; the complete logs are in Appendix 4.

```
$ sudo dnf reinstall -y systemd-boot-unsigned
...
Running scriptlet: libdnf5-plugin-actions (post_transaction)
[tboot-sbloader-posttrans] systemd-boot-unsigned changed -> invoking helper
[tboot-sbloader-helper]   sentinel set: /var/lib/tboot-sbloader/UNSAFE-T0-REBOOT
[tboot-sbloader-helper]   sbsign --key db.key --cert db.crt systemd-bootx64.efi ... ok
[tboot-sbloader-helper]   sbverify --cert db.crt (loopback both targets) ... verified
[tboot-sbloader-helper]   bootctl --no-variables install (propagate to ESP) ... ok
[tboot-sbloader-helper]   baseline advanced; sentinel cleared
Complete!
$ sbverify --list /boot/efi/EFI/systemd/systemd-bootx64.efi
signature 1: subject: /CN=tboot-lab db ... (custom db key)
```

Critical note on the final result. The implementation is technically operational in the test environment, but two limitations must be read alongside it. First, the trust model relies on the environment in which UKIs are signed. An attacker with root access to that environment at signing time can cause a malicious UKI to be signed by the legitimate chain. The solution therefore relocates trust to the signing environment rather than eliminating it, which is why centralized key management in Recommendation 4 is the logical next step. Second, the five example output blocks above illustrate the evidence format; the complete logs and hash checks with dates and timestamps are stored in Appendix 4 and should be independently verified there. The result is a validated laboratory-grade technical foundation, not a finished production product.

3.2 Conclusions

Answer to the research question. The shared principal question asked whether Piros could design, implement, and test a hardware-bound trusted-boot system based on TPM, UKIs, and Policy-Based Key Release so that only correctly booted Linux systems gain access to encrypted data. For the part assigned to the student - the signing chain, PCR measurements, and update cycle - the answer is affirmative. The architecture operates on Fedora Server 43 in Secure Boot enforcing mode; a kernel-install hook automates the update path without operator intervention; and runtime PCR 11 values match, byte for byte, the values predicted and signed during the build.

Justification of architectural choices. A kernel-install hook was selected for the update cycle instead of an image-based bootc approach because manual RPM-based update cycles remain much more common in the customer environments Piros is likely to encounter. The hook fills an operational gap between fully image-based operating systems and fragile manual re-enrollment. For forward sealing, the enter-initrd phase was selected because systemd-cryptsetup requests LUKS unlock at that stage. An earlier phase would add no relevant measurement, while a later phase would leave a window in which the initramfs is already running before the signed state is validated.

Principal finding. The most important surprise was that several gaps in upstream tooling documentation only become visible at firmware level rather than through Linux-side validation. A UKI accepted as correctly signed by sbverify can still be rejected by UEFI LoadImage(). The practical

conclusion is that a real boot in the target environment is the only reliable final validation step for a signed UKI. Linux-side tools remain useful, but they are not sufficient.

Limitations of the answer. Several claims are stronger in the laboratory than in production. Byte-for-byte agreement between predicted and runtime PCR 11 was demonstrated with a software TPM in a Proxmox VM and a fixed boot environment. On physical hardware, microcode updates, external firmware modules, and Option ROMs can alter PCRs 0 through 7, and PCR 11 may be affected by additional environmental factors. The kernel-install hook automates the package-managed kernel path and, through the fail-closed DNF automation chain, the dracut-sensitive path. Customer-specific package flows such as NVIDIA akmod/kmod and transactions that introduce a new kernel version were not yet covered in the lab. The objcopy failure analysis is plausible under the PE+ specification but was not proven down to the precise differing field; a formal upstream bug report would require deeper analysis. These limitations do not invalidate the main answer, but they qualify it: the architecture works at laboratory scale and is technically ready for production-oriented extension.

Value to the workplace. The project directly supports Piros's position as an enterprise open-source and Red Hat consultancy. Trusted boot on Linux is an emerging technology. By developing it now, Piros can understand its operation before customers request it and before it appears as a complete feature in RHEL. The project delivers two reusable assets: a reference implementation and a reproducible runbook that another consultant can take over or reproduce at a customer without depending on the original author. An evidence-based risk map records which attacks are blocked, which remain outside scope, and which require further validation. This gives Piros a foundation for a future service offering for customers with sensitive Linux infrastructure. The delivered solution remains a validated technical basis, not a production-ready product.

Four issues remained open for customer deployment at submission time: validation on physical hardware with a physical TPM and actual firmware rather than EDK II; validation against customer-specific package flows such as NVIDIA akmod and kmod and kernel-upgrade transactions; an operational policy for firmware updates after removing factory keys from the chain of trust; and centralized key management through an HSM or remote signing service as production hardening. These points are included in the recommendations. The remaining work does not reduce the existing value: the architecture can be extended without rebuilding the current implementation.

3.3 Recommendations

Four recommendations are made for Piros on the basis of the project. The first two extend the existing architecture technically. The third and fourth concern operational matters outside the student's assigned scope that are required before the solution can be considered production-ready.

Recommendation 1: validate the dracut-only update path against customer package flows. The kernel-install hook covers kernel package updates. Packages such as NVIDIA drivers can trigger dracut without directly traversing kernel-install, so the signing hook is not called by that package alone [22]. The fail-closed DNF automation chain developed for Subquestion 8 closes this gap: the decider detects drift in monitored boot inputs, including dracut configuration and kernel modules, and calls the executor, which rebuilds and signs the UKIs through kernel-install and the hook. The remaining recommendation is therefore not to build this mechanism, but to validate it against real customer

package flows, especially third-party akmod/kmod workflows and the kernel-upgrade path that introduces a new kernel version.

Recommendation 2: broaden validation of the automated systemd-boot signing path. A systemd package update replaces the systemd-boot loader binary, which must then be signed again with the custom db key. This path is automated and was validated in the lab through tboot-sbloader-posttrans and tboot-sbloader-helper, which re-sign the binary and propagate it to the ESP. The next step is validation against future systemd package updates, physical firmware implementations, and customer environments.

Recommendation 3: establish an operational firmware-update policy. Removing factory keys from the Secure Boot hierarchy also blocks the standard fwupd path because vendors sign firmware capsules with keys that are no longer present in db [28]. A pragmatic operating procedure is to stage firmware updates on an isolated machine that retains factory keys, validate them there, and introduce the validated result to production through a controlled process. This is not a technical defect in the boot chain but an operational requirement that must be designed and documented before broad deployment.

Recommendation 4: move signing keys toward centralized key management. In the current setup the policy private key is sealed by the TPM and the db key is stored locally. For customer use, the architecture can evolve toward a central signing service backed by a Hardware Security Module or remote signer, ensuring that private keys never leave central infrastructure. This is outside the scope of the graduation project but fits the existing hook architecture: only the signing operation must be redirected to the external service.

Possible improvements to the graduation project. A formal structural PE+ analysis of the failed objcopy UKI, using a PE parser or hexadecimal comparison with a working UKI, could have strengthened the root-cause analysis from a plausible explanation to proof of the exact field that differs. A second improvement would be an upstream bug report with reproduction steps for the silent ukify build --join-pcrsig failure. These activities lie beyond the scope of the submitted report and are natural follow-up work.

4 Personal Reflections

This chapter reflects on the graduation project using the AIM reflection model: What went well, What went less well, Insight, and Appreciation. Two reflections are included: the first on the technical process and the second on personal learning, linked to the X-factor and the program-specific learning outcomes (OLRs).

4.1 Personal Reflection

4.1.1 AIM Reflection 1: The Technical Process

What went well. The strongest element was the strict stage-gating and snapshot discipline. A Proxmox snapshot was taken before every irreversible step, including firmware key enrollment, TPM sealing of the policy private key, and the first live update through the hook. This made it possible to conduct genuinely risky experiments without endangering overall progress. When a step failed, recovery was a single rollback command rather than a day of repair work. A second successful choice was to write the

runbook reproducibly: instead of entering fixed hashes manually, the scripts read the relevant values dynamically. This required extra effort during development but makes the documentation useful for much longer.

What went less well. The gap between theory and practice was too large during the early part of the project. Too much reading and documentation took place before the first line of code was tested in a VM. As a result, several categorical assumptions drawn from documentation - for example, that `objcopy --add-section` was sufficient to add `.pcrsig` to a UKI - were only disproved during implementation. In retrospect, a simple build-sign-boot cycle should have been established earlier, with the architecture expanded only after that base path worked. The design-first approach produced a thorough understanding, but at the cost of later debugging cycles that could have been avoided.

Insight. The principal lesson is that documentation gaps do not reveal themselves automatically. A tool reporting that a UKI is signed does not mean the firmware will accept it, and a command returning exit status 0 does not prove that it performed the intended work. Future verification should always occur at the lowest level at which the effect matters. For a boot chain, that means a real boot in the actual virtual machine rather than only a Linux-side check. A second insight was that cross-validating my own scripts with a second AI tool exposed two real defects. Independent verification has operational value even when it initially feels redundant.

Appreciation. The collaboration with fellow student Matteo Ideler Cautaert worked well: the formal division of responsibilities was precise, while the final documentation contained enough overlap to understand each other's work. Workplace coaches Merijn Lowis and Indy van Mol deserve explicit recognition for allowing the degree of independence required by the project and providing rapid technical feedback when requested. PXL supervisor Guido Frissaer also identified stylistic problems in Dutch technical writing that would otherwise have remained invisible to me.

4.1.2 AIM Reflection 2: Personal Development, X-Factor, and OLRs

What went well. I am satisfied with the extent to which I learned to master a complex subject without losing the overall structure. The project contains roughly ten layers of abstraction between UEFI firmware and the final LUKS unlock. A year earlier I would likely have stalled at the first layer where documentation contradicted observed VM behaviour. During this project, such contradictions became the starting point for investigation rather than a blocking problem.

What went less well. During the first months I found it difficult to ask for help at the right time. Work of this kind creates a temptation to solve everything independently before consulting a coach or colleague. Sometimes that is appropriate because the investigation itself produces learning. At other times it consumes hours that a short, focused question could have saved.

Insight. My largest personal learning gain is the realization that technical directness and respectful communication are compatible. In a DevOps context, a concise, evidence-based correction of an incorrect mental model is often the most respectful response, not a form of confrontation. In report writing, this means that clear language without unnecessary hedging serves the reader better than diplomatic wording that hides the actual conclusion.

Appreciation. The personal success milestone was the first reboot in which the kernel-install hook worked end to end: a DNF reinstall transaction produced a fresh signed UKI, the system selected that

UKI on the next boot, and runtime PCR 11 matched the hook's own prediction byte for byte. That result confirmed that the entire architecture, from the chain of trust through forward sealing, had been validated at the correct level. I am proud of that achievement.

Connection to the X-factor. The graduation project aligns most strongly with two X-factor pillars. The first is entrepreneurial and innovative: the solution does not merely repeat existing documentation, but fills an operational gap between three update paradigms and delivers a working reference implementation on a platform for which Piro's does not yet have a formal offering. The second is multidisciplinary work: the project spans firmware and PE+, Linux system administration, cryptography, and operational governance. None of those disciplines is sufficient on its own. A third contributing pillar is **passion**: extending the work beyond the original proposal to make the update path fully operational resulted from personal interest rather than a mandated requirement.

Connection to the OLRs. The project addresses several program-specific learning outcomes of the Associate Degree in Systems and Network Administration. The six most relevant outcomes are discussed below, together with the concrete action or result that demonstrates each one and, where appropriate, a critical observation.

OLR2 (Installer: clients, servers, and peripherals in a professional virtual or physical environment). The complete test setup was implemented as a Fedora Server 43 VM in Proxmox VE with a software TPM for cryptographic measurements. The base operating system was installed and configured, and a snapshot-based method made every irreversible action - including firmware key enrollment and TPM sealing - reproducible. This demonstrated that a professional virtual environment is more than a VM that happens to run: without snapshot and recovery discipline, several failures in this project would have required substantial repair instead of a rollback lasting seconds.

OLR3 (Installer: operating systems, applications, and network services). Operating-system configuration went far beyond a standard Linux installation. A custom Secure Boot hierarchy of PK, KEK, and db keys was generated and enrolled in firmware; systemd-boot was signed with the custom db key; systemd-pcrphase services were configured in the correct boot-phase order; and the kernel-install pipeline was extended with a custom drop-in. This showed that operating-system configuration at this level requires simultaneous understanding of firmware, kernel build systems, and systemd internals.

OLR4 (Installer: identify security risks and apply system-security best practices). This outcome is central to the project. The security risk is the problem statement itself: conventional controls are ineffective when the boot chain has already been compromised. The resulting best practices include hardware-bound integrity measurements through TPM PCRs, forward-sealed PCR signatures renewed with every kernel update, and a stricter chain of trust without factory keys. At the same time, the project demonstrated that best practices are contextual. The same choices that secure the boot chain can block standard fwupd firmware updates. Maturity therefore lies not only in applying a recommendation, but in understanding why it cannot be introduced without operational coordination.

OLR6 (Administrator: automate repetitive tasks in systems and network infrastructure). The kernel-install hook is the clearest automation example in the project. Without it, each kernel update would require a manual procedure of five to ten steps: build the UKI, sign it, predict PCR 11, sign the prediction, place the UKI on the ESP, and set the default. The hook performs the complete sequence during a normal DNF transaction. This demonstrated that good automation does more than save time;

it excludes error paths. A manual operator can forget to sign or select the wrong forward-seal phase, while the hook makes those mistakes structurally impossible. A critical limitation is that robust failure handling was added relatively late; early scripts focused for too long on the success path alone.

OLR10 (Communicator: collect information, document, report, and use appropriate English technical terminology). Writing this report, maintaining the reproducible runbook, and creating the findings catalog all contribute to this outcome. Technical English terms such as UKI, PCR, forward sealing, Authenticode, and PE+ binary are unavoidable in this field. The project demonstrated that documenting while implementing, rather than reconstructing the work afterward, significantly reduces effort. Some findings now present in the catalog would have been difficult to reconstruct accurately three weeks later.

OLR14 (Lifelong learner: practice self-criticism, develop self-knowledge, and use it for personal and professional growth). The largest personal growth was not solely technical, although the technical development was substantial. It was learning to manage the tension between theoretical preparation and practical execution. My natural tendency is to understand everything before taking action. In a complex boot chain, that does not scale: too many layers contain differences between documentation and observed behaviour to reason through everything in advance. The self-critical lesson is to move to executable experiments earlier in future projects, even when theoretical understanding is incomplete. In a DevOps context, a short experiment often produces more learning than extended preliminary study.

4.2 Reflection on the Sustainable Development Goals

A trusted boot chain affects sustainable development in ways that are not immediately obvious. The following sections discuss three SDGs to which the project can contribute positively and three areas where unintended negative effects can arise.

4.2.1 Potential Positive Contributions

SDG 9 (Industry, Innovation and Infrastructure), target 9.1 on reliable and resilient infrastructure. The project delivers a working reference implementation of a boot architecture whose integrity can be verified at hardware level. For digital infrastructure supporting socially important services such as healthcare, public administration, and energy, this adds a layer of reliability beyond conventional security controls. A critical note is that SDG 9 often measures progress by volume - more infrastructure and connectivity - without placing equal emphasis on infrastructure quality and security. This project contributes more to a qualitative than quantitative interpretation of target 9.1 and supports the view that infrastructure investment is sustainable only when the resulting infrastructure is also secure.

SDG 16 (Peace, Justice and Strong Institutions), target 16.10 on access to information and protection of fundamental freedoms. Binding LUKS disk encryption to a verified boot state can protect journalists, activists, and organizations handling sensitive information from an attacker who gains physical access and silently installs a modified operating system. The technology turns traditional evil-maid attacks from invisible compromises into detectable failures. It also has a dual-use dimension: authoritarian actors can use the same mechanisms to restrict what software devices are allowed to run, while repair and reuse outside a factory trust chain become more difficult. This implementation mitigates that

concern by using owner-generated keys rather than factory keys, placing control of the chain of trust with the machine owner rather than the manufacturer.

SDG 12 (Responsible Consumption and Production), target 12.5 on reuse and life extension. A well-documented and reproducible boot architecture on commodity hardware can extend the useful life of existing machines because organizations do not automatically need to replace hardware in order to meet higher integrity requirements. This is the weakest positive SDG connection and depends heavily on implementation choices, as discussed under the negative impact on SDG 12.

4.2.2 Potential Unintended Negative Impact

SDG 12 (Responsible Consumption and Production): TPM-bound hardware and vendor lock-in. A TPM is physically tied to a machine. If a TPM-bound encryption key can no longer be released, for example after motherboard replacement or because the TPM generation becomes obsolete, the system may effectively be written off even when the remaining hardware is usable. This creates tension with target 12.5. An operational policy can make defective components difficult to replace independently without losing encrypted data. This is not an inevitable outcome, but it is a real risk when backup and recovery procedures are not designed carefully.

SDG 9 (Industry, Innovation and Infrastructure): barriers to repair, refurbishment, and circular reuse. A stricter chain of trust increases the practical difficulty of third-party repair and refurbishment. Repair companies, refurbishers, and reuse centers face additional complexity when Secure Boot key hierarchies must be cleared, regenerated, and redistributed. The PK/KEK/db lifecycle is not a consumer workflow. Enterprises with IT staff can manage it, but it introduces implicit cognitive and financial cost for the wider economic actors SDG 9 is intended to support.

SDG 10 (Reduced Inequalities): the digital-security divide. A trusted boot chain requiring a hardware TPM, UEFI firmware administration, and specialized upstream tooling is realistically accessible only to organizations that possess suitable hardware and can employ or contract the required expertise. This can raise the security level of large enterprises while smaller organizations, NGOs, and individual users in lower-income regions remain limited to basic controls. At system level, the technology can therefore widen rather than reduce the digital-security gap. Security technology has an equalizing effect only when it is widely accessible through open-source tooling on common hardware. The project's choice of Fedora and upstream systemd rather than a proprietary product moves in that direction but does not solve the structural problem.

General critical observation on the SDG analysis. The SDGs are often criticized because they do not address conflicts between goals strongly enough. This project illustrates that problem at small scale: the same technology that supports SDGs 9 and 16 can, under other implementation choices, undermine SDGs 9, 10, and 12. An honest reflection cannot simply add points to a score. It is a negotiation between objectives that can contradict one another. The value of the SDG exercise lies in making that tension explicit rather than forcing a uniformly positive conclusion.

5 References

The sources are ordered according to IEEE style. References in the text use the corresponding number in square brackets. Official upstream documentation and specifications from the UAPI Group, systemd, the Trusted Computing Group, the UEFI Forum, and cryptsetup form the primary basis for architectural decisions. Red Hat and Fedora vendor documentation was used for distribution-specific implementation details.

[1] systemd Project, "Unified Kernel Image (UKI) Specification," 2024. [Online]. Available: https://uapi-group.org/specifications/specs/unified_kernel_image/. [Accessed: May 10, 2026].

[2] systemd Project, "Boot Loader Specification," 2024. [Online]. Available: https://uapi-group.org/specifications/specs/boot_loader_specification/. [Accessed: May 10, 2026].

[3] systemd Project, "TPM2 PCR Measurements made by systemd," 2024. [Online]. Available: https://systemd.io/TPM2_PCR_MEASUREMENTS/. [Accessed: May 10, 2026].

[4] systemd Project, "ukify(1) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/ukify.html>. [Accessed: May 10, 2026].

[5] systemd Project, "kernel-install(8) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/kernel-install.html>. [Accessed: May 10, 2026].

[6] systemd Project, "systemd-measure(1) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-measure.html>. [Accessed: May 10, 2026].

[7] systemd Project, "systemd-pcrphase.service(8) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-pcrphase.service.html>. [Accessed: May 10, 2026].

[8] systemd Project, "systemd-cryptenroll(1) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-cryptenroll.html>. [Accessed: May 10, 2026].

[9] systemd Project, "systemd-creds(1) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-creds.html>. [Accessed: May 10, 2026].

[10] systemd Project, "systemd-stub(7) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-stub.html>. [Accessed: May 10, 2026].

[11] Unified EFI Forum, "UEFI Specification, Version 2.10," Aug. 2022. [Online]. Available: <https://uefi.org/specifications>. [Accessed: May 10, 2026].

[12] Microsoft Corporation, "PE Format," Microsoft Learn, 2024. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format>. [Accessed: May 10, 2026].

[13] Trusted Computing Group, "TPM 2.0 Library Specification, Family 2.0, Level 00, Revision 1.83," Jan. 2024. [Online]. Available: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>. [Accessed: May 10, 2026].

- [14] M. Broz, "LUKS2 On-Disk Format Specification, Version 1.1.1," cryptsetup project, 2023. [Online]. Available: https://gitlab.com/cryptsetup/cryptsetup/-/wikis/LUKS2-docs/luks2_doc_wip.pdf. [Accessed: May 10, 2026].
- [15] cryptsetup project, "cryptsetup(8) Manual Page," 2024. [Online]. Available: <https://man7.org/linux/man-pages/man8/cryptsetup.8.html>. [Accessed: May 10, 2026].
- [16] Fedora Project, "Fedora Server Documentation," 2026. [Online]. Available: <https://docs.fedoraproject.org/en-US/fedora-server/>. [Accessed: May 10, 2026].
- [17] Red Hat, Inc., "Red Hat Enterprise Linux 9 Security hardening," 2025. [Online]. Available: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9/html/security_hardening/. [Accessed: May 10, 2026].
- [18] Free Software Foundation, "GNU Binutils objcopy Manual," 2024. [Online]. Available: <https://sourceware.org/binutils/docs/binutils/objcopy.html>. [Accessed: May 10, 2026].
- [19] sbsigntools project, "sbsigntools README," 2024. [Online]. Available: <https://git.kernel.org/pub/scm/linux/kernel/git/jejb/sbsigntools.git/>. [Accessed: May 10, 2026].
- [20] pesign project, "pesign GitHub Repository," 2024. [Online]. Available: <https://github.com/rhboot/pesign>. [Accessed: May 10, 2026].
- [21] efiteols project, "efiteols GitHub Repository," 2024. [Online]. Available: <https://github.com/vathpela/efiteols>. [Accessed: May 10, 2026].
- [22] dracut project, "dracut.modules(7) Manual Page," 2024. [Online]. Available: <https://man7.org/linux/man-pages/man7/dracut.modules.7.html>. [Accessed: May 10, 2026].
- [23] tpm2-software community, "tpm2-tools GitHub Repository," 2025. [Online]. Available: <https://github.com/tpm2-software/tpm2-tools>. [Accessed: May 10, 2026].
- [24] tpm2-software community, "tpm2-tss GitHub Repository," 2025. [Online]. Available: <https://github.com/tpm2-software/tpm2-tss>. [Accessed: May 10, 2026].
- [25] OpenSSL Project, "OpenSSL Documentation," 2025. [Online]. Available: <https://docs.openssl.org/>. [Accessed: May 10, 2026].
- [26] M. Garrett, "Building a more secure boot environment," mjpg59.dreamwidth.org, 2022. [Online]. Available: <https://mjpg59.dreamwidth.org/>. [Accessed: May 10, 2026].
- [27] Proxmox Server Solutions GmbH, "Proxmox VE Wiki: Nested Virtualization," 2025. [Online]. Available: https://pve.proxmox.com/wiki/Nested_Virtualization. [Accessed: May 10, 2026].
- [28] Linux Foundation Fwupd Project, "fwupd Project Documentation," 2025. [Online]. Available: <https://fwupd.org/>. [Accessed: May 10, 2026].
- [29] RPM Software Management Project, "Actions Plugin," DNF5 Documentation, 2025. [Online]. Available: https://dnf5.readthedocs.io/en/latest/libdnf5_plugins/actions.8.html. [Accessed: May 30, 2026].
- [30] systemd Project, "systemd-cryptsetup@.service(8) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-cryptsetup@.service.html>. [Accessed: May 30, 2026].

- [31] systemd Project, "crypttab(5) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/crypttab.html>. [Accessed: May 30, 2026].
- [32] systemd Project, "systemd-pcrlock(8) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-pcrlock.html>. [Accessed: May 30, 2026].
- [33] systemd Project, "systemd-tpm2-setup.service(8) Manual Page," 2025. [Online]. Available: <https://www.freedesktop.org/software/systemd/man/latest/systemd-tpm2-setup.service.html>. [Accessed: May 30, 2026].
- [34] bootc Project, "bootc Documentation: Introduction," 2025. [Online]. Available: <https://bootc-dev.github.io/bootc/>. [Accessed: May 30, 2026].
- [35] CoreOS Project, "rpm-ostree: A True Hybrid Image/Package System," 2025. [Online]. Available: <https://coreos.github.io/rpm-ostree/>. [Accessed: May 30, 2026].
- [36] J. Rutkowska, "Evil Maid Goes After TrueCrypt!," The Invisible Things Lab, Oct. 2009. [Online]. Available: <https://theinvisiblethings.blogspot.com/2009/10/evil-maid-goes-after-truecrypt.html>. [Accessed: May 30, 2026].
- [37] J. Boone, "TPM Genie: Interposer Attacks Against the Trusted Platform Module Serial Bus," NCC Group, 2018. [Online]. Available: <https://github.com/nccgroup/TPMGenie>. [Accessed: May 30, 2026].
- [38] Dolos Group, "From Stolen Laptop to Inside the Company Network," 2021. [Online]. Available: <https://dolosgroup.io/blog/2021/7/9/from-stolen-laptop-to-inside-the-company-network>. [Accessed: May 30, 2026].
- [39] rhboot project, "shim: UEFI Secure Boot first-stage bootloader," 2024. [Online]. Available: <https://github.com/rhboot/shim>. [Accessed: May 30, 2026].
- [40] rhboot project, "SBAT: Secure Boot Advanced Targeting," 2024. [Online]. Available: <https://github.com/rhboot/shim/blob/main/SBAT.md>. [Accessed: May 30, 2026].
- [41] Microsoft Corporation, "Windows Authenticode Portable Executable Signature Format," 2008. [Online]. Available: https://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/Authenticode_PE.docx. [Accessed: May 30, 2026].
- [42] K. Moriarty, Ed., B. Kaliski, J. Jonsson, and A. Rusch, "RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2," Internet Engineering Task Force, Nov. 2016. [Online]. Available: <https://www.rfc-editor.org/info/rfc8017>. [Accessed: May 30, 2026].
- [43] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, "RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," Internet Research Task Force, Sep. 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9106>. [Accessed: May 30, 2026].
- [44] L. Poettering, "Brave New Trusted Boot World," Opointer.net, Oct. 2022. [Online]. Available: <https://Opointer.net/blog/brave-new-trusted-boot-world.html>. [Accessed: May 30, 2026].
- [45] The Linux Kernel Organization, "TPM Security," The Linux Kernel Documentation, 2024. [Online]. Available: <https://docs.kernel.org/security/tpm/tpm-security.html>. [Accessed: May 30, 2026].

6 Appendices

This section lists the numbered appendices that support correct reading and interpretation of the report. The appendices were submitted separately and are available for inspection at the workplace.

- **Appendix 1:** Build-time architecture diagram. Steps from dnf upgrade kernel to the signed UKI on the ESP, including the kernel-install hook, unsealing of the policy private key from the TPM, and the three-way verification procedure.
- **Appendix 2:** Boot-time architecture diagram. UEFI firmware verifies the Secure Boot signature, systemd-stub measures UKI sections into PCR 11, systemd-pcrphase services extend PCR 11 for each phase, and systemd-cryptsetup unlocks LUKS through the PCR 11 policy.
- **Appendix 3a:** Complete source code of the kernel-install hook at /etc/kernel/install.d/80-tpm2-sign.install, including version and cryptographic hash.
- **Appendix 3b:** Complete source code of the first fail-closed UKI automation chain: tboot-dnf-posttrans, tboot-dnf-helper, the tboot-predict-pcr11 predictor, and the 50-tboot-posttrans.actions DNF rule, each with version and cryptographic hash.
- **Appendix 3c:** Complete source code of the second fail-closed systemd-boot-loader automation chain: tboot-sbloader-posttrans, tboot-sbloader-helper, and the 60-tboot-sbloader.actions DNF rule, each with version and cryptographic hash.
- **Appendix 4:** CLI output for runtime validation. PCR 7 and PCR 11 readings from /sys/class/tpm/tpm0/pcr-sha256/ compared with the systemd-measure calculate prediction, including the byte-for-byte match.
- **Appendix 5:** Diagnostic Findings Catalog. Findings organized across multiple categories, each with a verdict - FORBIDDEN, CONDITIONAL, or CONTEXT-ONLY - a symptom, root-cause analysis, and fix.
- **Appendix 6:** Results of the attack scenarios. For each scenario - modification of vmlinuz, initramfs, or the command line; UKI replacement; and rollback to an older signed UKI - an execution log and short analysis identify the stage at which the boot chain rejects the attack.
- **Appendix 7:** Forensic artifact bad-objcopy-pcrsig-loadererror.efi. The failed UKI is accepted as correctly signed by sbverify but rejected by UEFI LoadImage(). It remains available at the workplace for further analysis.